FireReady AR: technical documentation

Author	Mutabazi Ishimwe Bruno	Platform	Android (ARCore), API 26+, ARM64
Assignment	Summative Assignment 2, AR simulation	Engine	Unity 6000.4.7f1, URP, AR Foundation 6.4
Repository	github.com/mutabazi-bruno/FireReady-AR-Simulation-Technical-Implementation		

1. What I built

FireReady AR is a fire extinguisher training app. The user places a small workshop office on their floor with AR Foundation. Three fires (a paper bin, a fuel spill and an overloaded extension board) ignite at 0, 20 and 40 seconds. To pass, the user has to raise the alarm, pick the right one of four extinguishers for each fire, pull the pin, spray the base from 1 to 2.5 m away, switch off the power before tackling the electrical fire, and evacuate if a fire gets out of control. The session ends with a score out of 100 and a list of mistakes, and it is saved on the phone.

The app implements the single scenario defined in my Assignment 1 SRS (Workshop office fire) and its requirements FR-01 to FR-32. The code is 94 C# scripts (about 7,600 lines) in one assembly, `FireReady`, plus an EditMode test assembly. The rest of this document explains how it is put together and where it differs from the original design.

2. Final architecture

I kept the four layers from the design. Each system is a MonoBehaviour on a child of the `Systems` object in `Main.unity`, and they are connected through serialized references in the Inspector rather than by searching the scene at runtime.

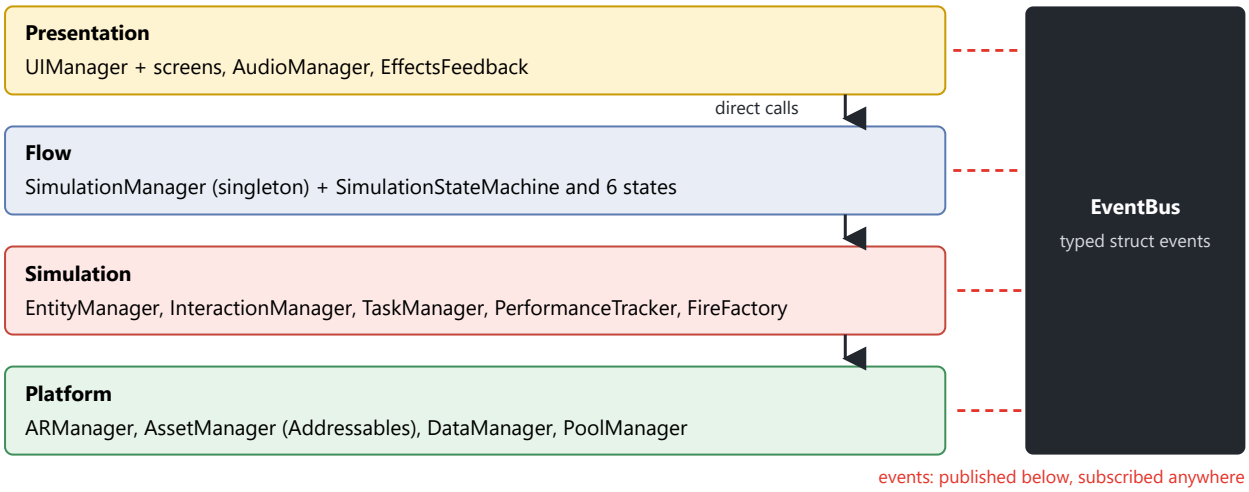


Figure 1. Layers and communication. Calls go downwards only; anything that needs to travel upwards or sideways is an event.

The `SimulationManager` owns the session state machine. Its states call the systems they need on Enter and Exit: `ARSetupState` starts plane scanning, `RunningState` ticks the entities, interaction and task systems each frame, and `SummaryState` saves the session. Lower layers never hold a reference to the UI or audio. When a fire goes out, `FireSource` publishes `FireExtinguished`, and the task list, score, HUD toast, steam effect and sound all react on their own.

The main loop is short. `InteractionManager` casts a ray from the centre of the camera every frame (`Physics.RaycastNonAlloc`) to find the targeted fire and whether the ray hits its base collider. While the squeeze button is held, the held `Extinguisher` asks its agent how it works on that fire class and calls `FireSource.ApplyAgent`. The fire changes its intensity, publishes `AgentApplied`, and updates its state from the thresholds (35 and 75). `PerformanceTracker` turns these events into mistakes and aim accuracy, and `ScoreCalculator` applies the deductions from Table 2.2 of the SRS.

3. OOP implementation

Encapsulation. A fire's intensity is private and read through a property. The only ways to change it are `Tick()` (growth) and `ApplyAgent()` (suppression), so nothing can set a fire to zero and skip the rules. An `Extinguisher` hides its remaining agent and pin state behind `PullPin()` and its discharge methods, and refuses to spray with the pin in. Config ScriptableObjects are read-only at runtime, and only `PerformanceTracker` writes to the session record.

Abstraction. Systems depend on small interfaces. `IInteractable` lets `InteractionManager` handle any tappable object. `IExtinguishingAgent` hides how an agent reacts to a fire class. `ISimulationState` hides what each stage does. `IDataStorage` hides where history is stored, which is what lets the tests use an in-memory storage instead of a real file. `IPoolable` is what the pool needs from an object (`OnSpawn`, `OnDespawn`).

Inheritance. `SimulationEntity` is the abstract base for everything in the scene that the user can interact with. `AlarmCallPoint`, `PowerBreaker`, `EmergencyExit`, `Extinguisher` and the abstract `FireSource` derive from it. `OrdinaryFire`, `FlammableLiquidFire` and `ElectricalFire` derive from `FireSource`, and the four agents derive from the abstract `ExtinguishingAgent`. I kept the trees shallow (two levels at most) and only used inheritance where the subclasses really share state and behaviour.

Polymorphism. `OnTap()` is abstract, so tapping the breaker cuts the power while tapping the exit ends the session, and `InteractionManager` never checks the type. `FireSource` has virtual hooks (`OnDangerousHit`, `OnExtinguished`, `Tick`). `ElectricalFire` overrides `OnExtinguished` to start an 8 second reignition timer while the power is live, and `FlammableLiquidFire` overrides `OnDangerousHit` to spread a second fire through the factory when it is hit with water.

4. Design patterns

Pattern	Where	Problem it solves
State	`SimulationStateMachine`, `MainMenuState`, `BriefingState`, `ARSetupState`, `RunningState`, `PausedState`, `SummaryState`	Input, per-frame updates and the visible screen all depend on the stage. Each state owns its Enter, Tick and Exit, so pausing is just a state change: `PausedState` doesn't tick the entities, which freezes the fires (FR-23). The fire lifecycle stays an enum with thresholds, as planned, because those states only change data.
Strategy	`IExtinguishingAgent`, `WaterAgent`, `FoamAgent`, `CO2Agent`, `DryPowderAgent`	The result of a spray depends on the agent and the fire class. Each agent returns Effective, Weak, Dangerous or Fatal from `Evaluate(fireClass, powerLive)`, so `Extinguisher` and `FireSource` contain no agent-specific code. A fifth extinguisher type needs one class and one asset (NFR-10).
Factory	`FireFactory`	The `EntityManager` shouldn't know about prefabs or how each fire subclass is configured. The factory registers a prefab and `FireProfile` per class, creates and configures fires from the spawn data, and is reused by `FlammableLiquidFire` when it spreads.
Observer	`EventBus` with 41 event structs in `GameEvents.cs`	One action needs several reactions. Publishers don't know their subscribers, and subscribers unsubscribe in `OnDisable()` so destroyed objects never get events.
Object pool	`ObjectPool<T>`, `PoolManager`, `PooledEffect`, `PooledAudioSource`	Spraying creates several short-lived effects and sounds per second. Creating and destroying them causes garbage collection spikes on a phone (NFR-01).
Singleton	`SimulationManager` only	Exactly one object controls the session and every screen needs to reach it. The other systems are reached through it or through Inspector references, so dependencies stay visible.

5. Event system

`EventBus` is a static class holding a `Dictionary<Type, Delegate>`. `Subscribe<T>` and `Unsubscribe<T>` combine and remove delegates, and `Publish<T>` invokes them. Every event is a `struct` with a `where T : struct` constraint, so events published every frame, like `AimUpdated` and `AgentApplied`, don't allocate. Because the bus is static, it is cleared with `[RuntimeInitializeOnLoadMethod]` at startup so stale handlers can't survive between play sessions in the Editor. State changes are published before the new state's `Enter()` runs, so the UI has already switched screens when the state starts its work.

6. Object pool

`ObjectPool<T>` wraps `UnityEngine.Pool.ObjectPool` for any `Component` that implements `IPoolable`. `Get()` activates the object, places it and calls `OnSpawn()`. `Release()` calls `OnDespawn()`, deactivates it and puts it back under the pool's parent. `PoolManager` creates one pool per effect (spray puff, steam burst, splash, smoke puff) from the prefabs in the Effects group, plus a pool of one-shot audio sources, and pre-warms them when the scenario loads so the first spray doesn't instantiate anything. A `PooledEffect` clears and replays its particle systems on spawn and returns itself when its lifetime runs out. On despawn it stops and clears the particles and resets its scale, so a reused puff looks new. When the user returns to the menu, the pools are cleared before the Addressables handles are released.

7. AR implementation

`ARManager` wraps AR Foundation. While scanning, it watches `ARPlaneManager.trackables`, counts only `HorizontalUp` planes, and reports progress against the 1.5 m minimum (FR-04). A tap is raycast with `ARRaycastManager` against `PlaneWithinPolygon`, and the preview is rotated to face the user (FR-05); tapping again moves it (FR-07). On confirm, `ARAnchorManager.TryAddAnchorAsync` creates an anchor, the scenario root is parented to it, and the plane overlay is hidden (FR-06). Pause offers Reposition, which returns to placement and keeps the same session. If tracking is lost during a session, `ARManager` publishes `TrackingLost` with a reason, the simulation pauses and an overlay explains what to do (FR-24). `LightEstimation` applies ARCore's brightness and colour estimate to the scene light, so the fires and furniture are lit like the real room. In the Editor, with no AR provider running, the same code uses a virtual floor and a WASD camera, which is how I tested most of the gameplay.

8. Data persistence

This follows the data management plan from Assignment 1. `DataManager` creates a `SessionRepository` over a `JsonFileStorage` pointing at `sessions.json` in `Application.persistentDataPath`. The repository reads the file once at startup into a `SessionHistory` (with a `schemaVersion`), keeps the newest 50 records, and offers `Save`, `LoadRecent` and `Clear`. `JsonFileStorage.Write` writes to `sessions.json.tmp` and then swaps it in with `File.Replace`, so a crash mid-write leaves the old history intact (NFR-08). If the file can't be parsed, it is renamed to `.bak` and the app starts with an empty history instead of crashing. Each record holds the outcome, score, duration, fires extinguished, aim accuracy, alarm and power flags, agent usage and the list of mistakes with times. Saving happens when the summary opens, never during the simulation. Completed, failed, evacuated and abandoned sessions are all saved (FR-29), and the history screen lists them newest first and opens the full summary on tap (FR-30). Settings (volume, vibration, diagnostics) are kept in PlayerPrefs because they're a few small values.

9. Audio architecture

`AudioManager` only reacts to events. It looks up clips in an `AudioCueTable` ScriptableObject that maps each `AudioCueId` to a clip, volume, category and spatial blend. One-shots (pin pulled, breaker, wrong agent, task done, success and fail jingles) play through pooled audio sources at the event's position, so the alarm bell and the fire crackle come from where they are in the room. Long sounds use dedicated looping sources: the spray loop changes pitch per agent, each fire has a crackle loop that gets louder as it grows, and the alarm bell loops once

raised. The Core group holds the interface sounds and the scenario group adds the scenario cues when it loads, removing them again on release. Every sound has an on-screen text equivalent (a toast, the task panel or the distance chip) so nothing relies on audio alone (NFR-07), and serious mistakes also vibrate unless vibration is turned off.

10. Asset management

Group	Contents	Loaded	Released	Bundle size
Core	Audio cue table, briefing content	App start (`LoadCoreAsync`)	Never	67 KB
Scenario_WorkshopOffice	Layout prefab with models, 3 fire prefabs, 4 extinguisher configs, scenario config, scoring rules, scenario audio	After placement is confirmed	On returning to the main menu	25.1 MB
Effects	Pooled spray, steam, splash and smoke effects (label `effects`)	With the scenario	With the scenario, after the pools are cleared	5.8 MB

`AssetManager` keeps every `AsyncOperationHandle` it opens in a list. `LoadScenarioAsync` loads the scenario config first, then starts the layout, extinguisher, fire and effects loads together and waits on all of them, which keeps loading under the 3 second target in NFR-03. `ReleaseScenario` releases each handle once and clears the list. The order matters, so `SimulationManager.TeardownScenario` destroys the scene instances first, clears the effect pools, unregisters the scenario audio and only then releases the handles. Restart reuses the loaded scenario instead of loading it again. The Settings screen has a Diagnostics switch that shows the open handle count: 2 on the menu, more after placement, and back to 2 after returning to the menu, which is how I checked that every load has a matching release.

The scenario bundle was 91 MB at first, because glTFast embedded the `.glb` textures uncompressed. I re-exported the models as `.gltf` with separate texture files, which Unity imports and compresses itself with a 1024 px limit. That brought the bundle down to 25 MB and the APK to 71 MB.

11. Performance

- Effects and one-shot sounds are pooled and pre-warmed, so spraying doesn't instantiate or destroy anything.
- Events are structs and the aim ray uses `RaycastNonAlloc` with a reused hit buffer, so the per-frame path doesn't allocate.
- History is saved only when the summary opens and read once at startup, so the disk isn't touched during a session.
- Model textures are compressed and capped at 1024 px; the frame rate is capped at 60, the build is ARM64 with IL2CPP, and the URP asset is the mobile one.
- The cameras that draw the turning extinguisher on the menu and briefing only render while their screen is visible.

In the Editor, the diagnostics strip shows 59 to 61 FPS with the worst frame around 20 to 28 ms during a full session. The device measurements for NFR-01 and NFR-04 are shown on the Samsung phone in the demo video.

12. Testing

The EditMode tests (41 test cases) cover the full agent and fire class table, fire growth, thresholds, reignition after a weak agent and with live power, the off-base factor, every score deduction and the Failed cap, the event bus subscribe and unsubscribe rules, and the save file: a normal round trip, keeping only the newest records, and recovering from a corrupt file. I tested the full flow in the Editor with the virtual floor, and the Android build was made with 0 errors. The build manifest asks only for camera and vibrate permissions.

13. Deviations from the SRS

SRS / design said	What I built	Why
Core group holds UI sprites, fonts and interface sounds	Core holds the audio cue table and briefing content; UI sprites and fonts are in the main scene	The start screen has to show before any Addressables load finishes, so its art can't wait for a group.
Effects group holds the flame effects	Flames are part of each fire prefab in the scenario group; Effects holds the pooled short-lived effects	A fire's flames and smoke scale with its intensity and live as long as the fire, so they belong with it. Only pooled effects are shared.
AudioManager owns mixer groups	The master volume goes through `AudioListener.volume` and a volume per category (interface, effects, ambient) is applied to each source	There is one volume slider and no effects processing, so an AudioMixer asset added nothing.
Event `PlaneReady`	`PlaneScanUpdated` with size and progress, plus extra events such as `AimUpdated` and `DistanceWarning`	The setup screen shows a progress bar, which needs more than a ready flag.
Not in the SRS	Briefing extinguisher render, scorch marks and charring that grow with burn time, held nozzle and hose, AR light estimation, diagnostics overlay	Added for feedback and realism. None of them change the rules or the scoring.
Placeholder art in the design	Third-party models, textures, effects and sounds	Every asset, its source and license is listed in `CREDITS.md`; some models were reduced in Blender for the phone.
ARCore required	ARCore set to Optional	NFR-09 asks for a clear message on phones without ARCore. With Required, the store blocks the install, so the user never sees it.

14. Limitations and future improvements

- There is one scenario, as the SRS scoped it. A kitchen or server room scenario would be a new Addressables group and a new `ScenarioConfig`; the loading code wouldn't change.
- Fire behaviour is rule-based. It's correct for training decisions but it doesn't model heat, smoke spread or real discharge ranges.
- Aim is judged by a single ray from the centre of the screen, so sweeping only counts while the ray crosses the base collider. A cone check would match a real spray better.
- Anchor stability depends on ARCore and the floor texture. On plain shiny floors scanning is slow and the scene can drift.
- History stays on one phone. A trainer would need an export, for example to CSV, to compare several trainees.